

Group 11 Members: Craig Vande Ligt, Collin Potter-Bonar, Colin Steensma, Rob Mifsud, Gabe Wolofsky, Anca Rotaru

Topic: Prediction of Class A Vessel Location from Automatic Information System Streamed Data

Objective: To design a Big Data pipeline that ingests live AIS streamed data using Spark Structured Streaming, preprocess and clean the data, store it in a distributed format, and apply machine learning models to predict short-term Class A vessel movements, If time permits, the goal is to also apply the best model to the streamed data, mapping in near real-time the updated locations of these vessels.

Data Source and Details:

Live AIS Vessel Data Feed (WebSocket API)
<https://aisstream.io>

API: <wss://stream.aisstream.io/v0/stream>

Note: Data was collected using an incoming stream of data via Kafka over a one-hour period and stored as a parquet file. A downsampled dataset was then created of 500 Class A vessels to ensure adequate timelines for modelling and workflow development.

ais_clean_compacted.parquet records:	635,788	Zip file data size:	84.40 MB
ais_model_sample_single records:	11,095	Zip file data size:	1.96 MB

Attributes/Features:

- Message type
- Ship name
- Mmsi
- Time UTC
- Latitude
- Longitude
- Speed over Ground (SOG)
- Course over Ground (COG)
- True Heading
- Msg ID

Engineered Attributes/Features:

- Event Time
- N Points (how many data points there are for the given vessel)
- Time Delta to previous AIS message in seconds (dt_mean_sec)
- Time Delta to next AIS message in seconds (dt_next_sec)
- Latitudinal Lag (one per 10 historic data points) (lat_lag_X)
- Longitudinal Lag (one per 10 historic data points) (lon_lag_X)
- Speed Over Ground Lag (one per 10 historic data points) (sog_lag_X)
- Course Over Ground Lag (one per 10 historic data points) (cog_lag_X)
- Target Latitude (actual latitude of given vessel) (target_lat)
- Target Longitude (actual longitude of given vessel) (target_lon)

Methods:

Using the above API, a WebSocket protocol, and Kafka, the data was streamed into Spark over the period of one hour. During this time, per micro-batch, the data was preprocessed and cleaned to ensure ease of use later on. The dataset was then saved as a parquet file (*ais_clean_compacted.parquet*) to reduce the steps necessary for future work by the team. Features were then engineered to indicate changes over time, and, therefore, historic data points to ensure that the machine learning models could learn based on more than just the current observation.

Five hundred Class A vessels were then randomly selected to reduce the dataset size to a more manageable size, and subsequently saved as a parquet file (*ais_model_sample_single.parquet*).

Three sequential combination machine learning regression models (Random Forest, Gradient Boost Trees, and Linear/Elastic-Net) were individually created, trained and subsequently fitted to the assembler test dataset. Each model was trained on the latitudinal target using a hyperparameter-tuned cross-validation with three folds and an accuracy metric of Root Mean Square Error (RMSE) in degrees, and then transformed using the assembler test dataset. Each model was then trained on the longitudinal target using the same training methodology above, but was then transformed on the latitudinally transformed assembler test dataset, ensuring a sequential combination of both the latitudinal and longitudinal models. The models were then evaluated using several metrics, such as RMSE, Mean Absolute Error (MAE), Mean Absolute Percentage Error (MAPE), Maximum Error (Max), Median Error (Median), and the coefficient of determination (R-squared or R^2), and the best model (Linear/Elastic-Net) was saved and put into production. The dataframe was also updated and output as *ais_model_sample_lr_preds_single.parquet*.

Due to time constraints and the complexity of using lag features with streamed data (requiring historic data of at least 9 data points per vessel for inference and mapping), a stream-simulated architecture was utilized in which static historical data was treated as a stream of incoming events.

The actual locations of Class A vessels were then mapped using leafmap from the *ais_model_sample_lr_preds_single.parquet* dataset. Given more time, mapping of the actual and predicted vessels' locations with a rolling most recent 10 data points per vessel would have been implemented on streamed data.

Analysis:

The Random Forest model achieved high R^2 values (0.9996 for latitude and 0.9997 for longitude), indicating a strong overall fit. However, its distance-based errors were relatively large. The model produced an RMSE of approximately 113.6 km, a median error of 8.86 km, and a maximum error of 22.77 km. While many predictions were accurate, the model occasionally made large errors, reducing its reliability.

The GBT model performed worse overall. Although it also showed high R^2 values (0.9990 for latitude and 0.9989 for longitude), its errors were significantly higher. The GBT model had an RMSE of about 199.3 km, a median error of 27.67 km, and a maximum error of 47.84 km.

These results indicated that GBT struggled to consistently model short-term vessel movement in this dataset.

In contrast, the Linear/Elastic-Net model produced substantially better results across all metrics. It achieved very low RMSE values (0.0080 for latitude, 0.0121 for longitude, and 1.25 km overall). The median distance error was only 0.31 km, and the maximum error remained below 0.19 km. These results showed that the Linear/Elastic-Net model was both highly accurate and very consistent, making it the best-performing model in our analysis.

Conclusions:

The team's analysis concluded that the Linear/Elastic-Net model easily outperformed both the Random Forest and Gradient Boosted Trees Regression models. While the tree-based models showed strong R^2 values, they produced much larger distance errors and occasional extreme mistakes. This made them less reliable for predicting vessel locations, even though they appeared to fit the data well overall.

The Linear/Elastic-Net model worked best because short-term vessel movement is relatively smooth and predictable, especially when recent location, speed, and direction are included as features. Our feature engineering captured this behavior effectively, allowing a simpler linear model to make accurate predictions without overfitting. With a median error of just over 300 meters and very small maximum errors, Linear/Elastic-Net provided the most accurate and stable results.

This project demonstrated that simpler models can outperform more complex ones when the data is well prepared. It also reinforced the importance of using multiple evaluation metrics rather than relying on R^2 alone when selecting a model for real-world, streaming data applications.

The team also learned the complexity of maintaining and capturing streaming data for both machine learning model development and implementation. Near real-time mapping of the actual data also proved challenging. Rather than implementing the model to streamed data and mapping in near real-time, due to time constraints and the complexities involved with maintaining a stream of data and mapping in near real-time, mapping was performed on the *ais_model_sample_single.parquet* file, showing the locations of vessels. If the team had another week of time, we'd have liked to finalized the stream of data, apply the model, and map both the actual and predicted vessel locations, using the last 10 messages per vessel as a rolling streamed map.